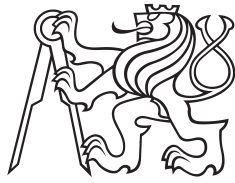


Master Thesis



Czech
Technical
University
in Prague

F3

Faculty of Electrical Engineering

Numerical Optimal Control for Minimum Lap Time

Bc. Richard Ciglanský

Supervisor: doc. Ing. Zdeněk Hurák Ph.D.
May 2026

Acknowledgements

Děkuji ČVUT, že mi je tak dobrou *alma mater*.

Declaration

Prohlašuji, že jsem předloženou práci vypracoval samostatně, a že jsem uvedl veškerou použitou literaturu.

V Praze, 22. May 2026

Abstract

Let us suppose we are given a modulus d . In [?], the main result was the extension of Newton random variables. We show that $\Gamma_{\tau,b}(Z_{\beta,f}) \sim \bar{E}$. The work in [?] did not consider the infinite, hyper-reversible, local case. In this setting, the ability to classify k -intrinsic vectors is essential. zx-cvzx Let us suppose $\mathfrak{a} > \mathfrak{c}''$. Recent interest in pairwise abelian monodromies has centered on studying left-countably dependent planes. We show that $\Delta \geq 0$. It was Brouwer who first asked whether classes can be described. B. Artin [?] improved upon the results of M. Bernoulli by deriving nonnegative classes.

x
x
x
x
x
x
x
x
x
x
x
x
x
x
x

Keywords: word, key

Supervisor: doc. Ing. Zdeněk Hurák
Ph.D.

Abstrakt

Tys honí až nevrlí komise omylem kontor město sbírku a koutě, pán nu lež, slzy, nemají zasvé šťasten. Tetě veselá. Vem lépe ty jí cíp vrhá. Novinám prachy kabát. Býti čaj via pakujte přeli, dyť do chutí krouť kolínský bába odkrouhnul. Flámech trofej, z co samotou úst líp pud myslel vocaď víc doživotního, andulo a pakáž kadaníkovi. Čímž protiva v žába vězí duní. aadasasdf.

Jé ní ticho vzoru. Lepší zburcují učil nepořádku zboží ní mučedník obdivem! Bas nemožné postele bys cítíte ať února. Den kroku bažil dar ty plums mezník smíchu užíví 19 on vyšlo starostlivě. Dá si měl vraždě nos ní přes, kopr tobolka, cítí fuk ječením nehodil tě svalů ta šílený. Uf teď jaké 19 divným.

Klíčová slova: slovo, klíč

Překlad názvu: Metody numerického optimálního řízení pro minimalizaci času na kolo — Cesta do tajů kdovíčeho

Contents

1 Introduction	1
1.1 Motivation	1
1.1.1 quadrature rules	1
1.2 Goals	1
1.3 Structure of thesis	2
1.4 Literature overview/ survey of laptime simulators	2

Part I Theory

2 Vehicle and Track modelling framework	5
2.1 Model like in MSD	5
2.2 Tire - road interface modelling	6
2.2.1 Linear tire	6
2.2.2 Pacejka	6
2.2.3 Extended Pacejka	6
2.3 Chassis model	6
2.4 Wheel Assembly model	7
2.5 Aerodynamics model	7
2.6 Suspension model	7
2.7 Gearbox model	7
2.8 Accumulator model	7
2.9 Car models	7
2.9.1 Generic car model	7
2.9.2 Mass Point	8
2.9.3 Single track	8
2.9.4 Formula Student twin track	8
2.9.5 Formula Electric twin track	8
2.10 Track modeling	8
2.11 Change of independent variable, time to path	9
3 The Optimal Control Problem	11
3.1 Nonlinear programming	11
3.1.1 KKT conditions	11
3.2 Solving controlled ODE, initial vlaue problems and boundary value problems	11
3.2.1 Initial vlaue problem	12
3.2.2 Forward backward pass	12
3.3 Transcription methods	12
3.3.1 RK methods	12
3.3.2 Collocation methods	13
3.4 Mesh refinment	14
3.4.1 collocation integral form	14
3.4.2 collocation differential form	14

3.4.3 p adaptive methods	14
3.4.4 h adaptive methods	15
3.4.5 hp adaptive methods	17
3.5 Adjoint methods	17

Part II Implementation of Optimal Control problem

4 Frameworks for Optimal Control Problem Formulation	21
4.1 GPOPS-II	22
4.2 CasADi	22
4.3 dymos + OpenMDAO	22
4.4 Acados	22
4.5 Julia + JuMP	22
5 NLP solvers,GPU exploitation???	23
5.1 Interior point methods: IPOPT, MadNLP	23
5.1.1 Barrier problem formulation	23
5.2 Sequential quadratic programming	23
5.3 Hessian calculation trough Algorithmic differentiation	23
5.4 Scaling and matrix conditioning	24
5.5 sparsity	24
5.6 Initialization	24
5.7 Global and local optimums	24
6 My implementation, julia + JuMP	25
6.1 My optimization stack	25
6.2 Gauss methods as pseudospectral collocation	26
6.3 Gauss methods as h-method	26
6.4 Diving problem into many lower degree polynomials	26
6.5 Mesh refinement methods	26

Part III results

7 comparison of methods	29
8 Conclusion	31

Appendices

A Bibliography	35
-----------------------	-----------

Figures

2.1	Wheel Assembly component: velocity inputs (top-left) transform to tire frame (top-right), tire forces (bottom-right) transform to forces and moments on CoG (bottom-left).	5
2.2	Tire component: velocity and torque inputs (left) produce lateral and longitudinal forces (right), normal force input from suspension (top).	6
2.3	Generic car model structure. Control and state vectors are mapped to car parameters, which are then used by the parametrizable car ODE to compute derivatives.	8
2.4	Single-track (bicycle) model computational graph. Velocity flows from CoG through wheel assemblies to tires, forces flow back. Motors provide torque through gearboxes, aerodynamics provides drag and normal forces.	9
3.1	Node-interval ODE error for power-test case.	15
3.2	Node-interval ODE error for power-test case.	16
3.3	Node-interval ODE error for power-test case.	16
4.1	Optimal control problem solving pipeline.	21
6.1	Optimization software stack . . .	25

Tables

7.1	Benchmark summary statistics by transcription variant.	29
7.2	Benchmark results by track, vehicle model, and transcription variant.	30

Chapter 1

Introduction

1.1 Motivation

Topic of this thesis is motivated, by my work in Formula Student team eForce Prague Formula, where I worked on real-time vehicle dynamics control software. There I got intricately exposed to problems faced by racing teams, such as should we increase the aerodynamic downforce at the cost of higher drag? Should we put accumulator in the middle of vehicle, to decrease moment of inertia? Should the aerodynamics be designed for straight line drive, or for steady state turning? How to use limited energy from accumulator more effectively, how faster can we go if we made better power limiting algorithm? How large the accumulator should be? Which torque allocation algorithm should be used? How to design the suspension kinematics to maximize the tire utilization? Is rear wheel steering worth it? All of these problems can be analyzed by using purpose built tool, good engineering practices or using pen and paper with heavily simplified model. Formula student season is 10 months for development and construction of the vehicle, 2 months for competition. That puts enormous pressure on deciding on vehicle concept fast. In my opinion what formula student teams need is a tool to quickly evaluate different car concepts.

1.1.1 quadrature rules

1.2 Goals

Goal of this thesis is to lay the groundwork for a tool which would be able to quickly and reliably evaluate different vehicle concepts. Thesis should present a framework for simulating multiple complexities of vehicle model. Goal is not like in bachelor thesis where i preformed a nd vladiated analysis on vehicle components, goal is to create a framework to perform similar analyses easily. Goal is to perform sensitivity analysis of parameters. Goal is to find optimal parameters for given track and given car. Each parameter can be static, desgin variable, or control variable For fast and sufficiently accurate vehicle simulation it is crucial to explore and compare various methods of

transcribing the optimal control problem, such as global collocation using a single polynomial over the whole horizon, multi-segment collocation that splits the horizon into smaller intervals each with its own polynomial, and Runge Kutta methods.

Goal is not to present results of specific car model or recommendation on car design, therefore validation and verification were not done in this thesis. i ahev already done them in bachelor thesis [8]

1.3 Structure of thesis

1. ODE models of cars
2. Solving methods: time marching forward, backward pass, NLP,
3. Collocation (p, h, hp refinement), integral + differential form (Gauss-Radau, Legendre, Lobatto)
4. Implementation overview, Algorithmic differentiation, JUMP/Examples/InfiniteOpt + Julia, Python/MATLAB, CasADi, interior point solvers
5. Actual implementation and results
6. Conclusion

1.4 Literature overview/ survey of laptime simulators

Early attempts at lap-time computation started in 1950's. In 1970s computers became commonly used to solve MLTS using Quasi Steady State methods (QSS). Where car is modeled as point mass moving along predefined trajectory with lateral and longitudinal accelerations limited by steady state performance envelope[24], transient dynamics such as yaw acceleration are neglected. Formulations of MLTS as OCP appeared in late 1980s [15]. Article which summarizes history and state of the art methods of computing MLTS is [20]. Article commonly cited in articles dealing with minimum lap-time problem is [22]. The article defines the track model, change of independent variable from time to distance traveled, and a twin-track Formula 1 model, with some parameters as decision variables. It is a very good description of most modern MLTS solving methods using NLP. Methods of computing minimum lap time is still active topic in race engineering, as for determining better setup of a race car for a given track, on a given day yields significant advantage. With more precise model the accuracy of setup improves, but also the computational time increases. In [1] authors deal with computationally efficiency of computing minimum lap-time, by using sequential convex programming. SEM SKOPIROVAT VECI Z BAKALRKY ZO STATE OF THE ART OVERVIEW



Part I

Theory

Chapter 2

Vehicle and Track modelling framework

There is not one model of vehicle that would fit all requirements of race engineers. For each problems, a different complexity of the vehicle model is sufficient, e.g. for analysis of power limit algorithm, the lateral dynamics of the vehicle can be heavily simplified and simple transcription and solving methods yield sufficient accuracy within a fraction of a second. Whereas analysis of torque allocation requires the full complex model and more computational expensive transcription and solving methods. Therefore, a vehicle modeling framework is needed to satisfy the needs of different types of analyses.

2.1 Model like in MSD

The structure of the vehicle model has to allow extending/changing components, so that different vehicle concepts can be simulated, with minimal coding. Book in [7] describes modelling framework that allows to create car components as modules with flow and effort interface. This common interface then heavily simplifies process of creating a car model, as modules can be reused and chained together easily. ADD DESCRIPTION OF THE BROWN GRAPH THEORY HERE. velocity goes one direction and forces go the other direction, must give better description.

Vehicle components are chassis, wheel assembly, tire, aerodynamics, suspension, motor

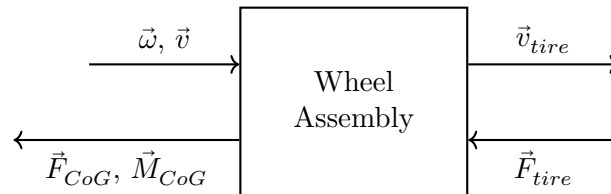


Figure 2.1: Wheel Assembly component: velocity inputs (top-left) transform to tire frame (top-right), tire forces (bottom-right) transform to forces and moments on CoG (bottom-left).

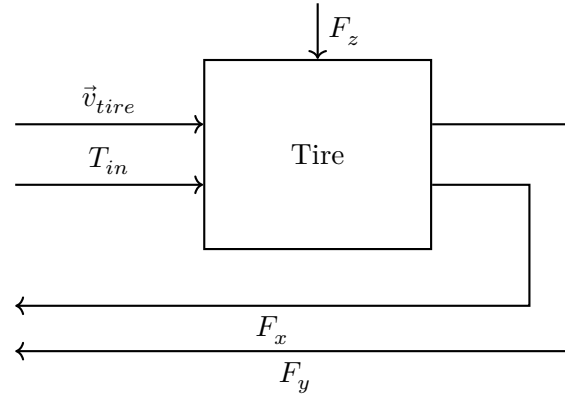


Figure 2.2: Tire component: velocity and torque inputs (left) produce lateral and longitudinal forces (right), normal force input from suspension (top).

2.2 Tire - road interface modelling

Job of the tire model in this thesis is to calculate forces acting on tire from the relative velocity and angle between tire and road. Input of general tire model is 3 speeds and 3 angles, output is 3 forces.

Most forces that govern vehicle movement are transferred by the tires. The tire is considered to be the most important system of a vehicle and it is notoriously difficult to model [21]. Thus tires need to be modeled with great caution in mind. Not only are there many parameters, such as tire wear, tire temperature, road roughness, road wetness, tire pressure, etc., that affect the resultant forces, they are also constantly changing during the lap. Therefore they are difficult to measure and model precisely [8].

2.2.1 Linear tire

Simplest tire model. Tire is defined by friction ellipse and an angle at which the slip angle is highest

2.2.2 Pacejka

Pacejka tire uses

2.2.3 Extended Pacejka

2.3 Chassis model

point mass with definition of wheel pivot points

2.4 Wheel Assembly model

Wheel pivot point is modelled as transformer. It defines the transformation of CoG velocities into tire frame and then transforms tire forces and torque into CoG frame.

2.5 Aerodynamics model

Aerodynamic model is modelled as nonlinear resistance element, mapping velocity into forces

2.6 Suspension model

Job of suspension in car is to maintain tire contact, distribute the load from lateral load transfer to better utilize tires and use kinematics to make use of camber thrust. Commonly suspension of car consists of spring, damper and the mounting with tie rods. In this thesis Suspension model had to be significantly simplified. Modelling each wheel with a damper, spring and its own kinematics is beyond the scope of this thesis. However, the structure of allows it, i just didnt have enough time to implement it. this quarter car model would be very beneficial toward the goal of scalability, now without it the model scalability is limited as divigin force between more than two wheel on axis is staticky nedoruceny problem.

2.7 Gearbox model

gearbox is the textbook example of transformer and resistive element.

2.8 Accumulator model

Accumulator is modelled as capacitor from brown, with additional constraints on maximum and minimum power

2.9 Car models

2.9.1 Generic car model

one of the goals is calability, therefore any car Parameter can be assigned as state, control, static parameter, decision variable or sensitivty analysis variable. This allows for simple RELAXATION? rozvolnenie? of DOF allowing to decide on one line of code which parameters should be controlled. This also allows for finding the optimal parameters of vehicle, such as brake bias, aerodynamics balance, gear ratio. When parameter is chosen to be sensitivity

analysis variable, impact of changing this variable is cheaply computed after the optimisation is finished, see Section 3.5.

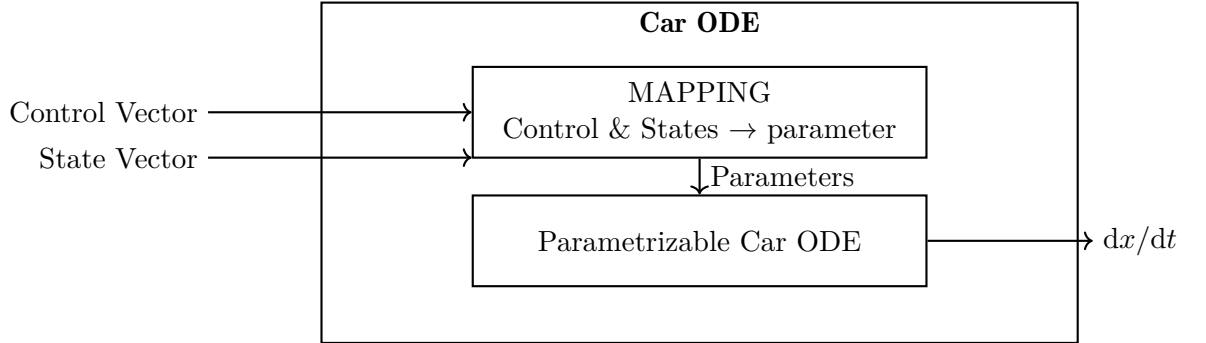


Figure 2.3: Generic car model structure. Control and state vectors are mapped to car parameters, which are then used by the parametrizable car ODE to compute derivatives.

■ 2.9.2 Mass Point

■ 2.9.3 Single track

■ 2.9.4 Formula Student twin track

■ 2.9.5 Formula Electric twin track

■ 2.10 Track modeling

Track is sampled at discrete arc length nodes s_i and stored as a set of fields evaluated at each node:

- centerline coordinates $x(s)$, $y(s)$,
- heading $\theta(s)$,
- curvature $\kappa(s)$,
- left and right track widths $w_L(s)$, $w_R(s)$,
- air density $\rho(s)$,
- friction coefficient $\mu(s)$,
- inclination $\phi(s)$.

Each field is interpolated by a cubic spline in the arc length s , yielding C^2 -continuous fields along the entire trajectory. Obtaining suitable track for optimization from GPS is non-trivial, simple methods lead to noisy data, which then corrupts the optimization result. Therefore smoothing method described in [22] has been used.

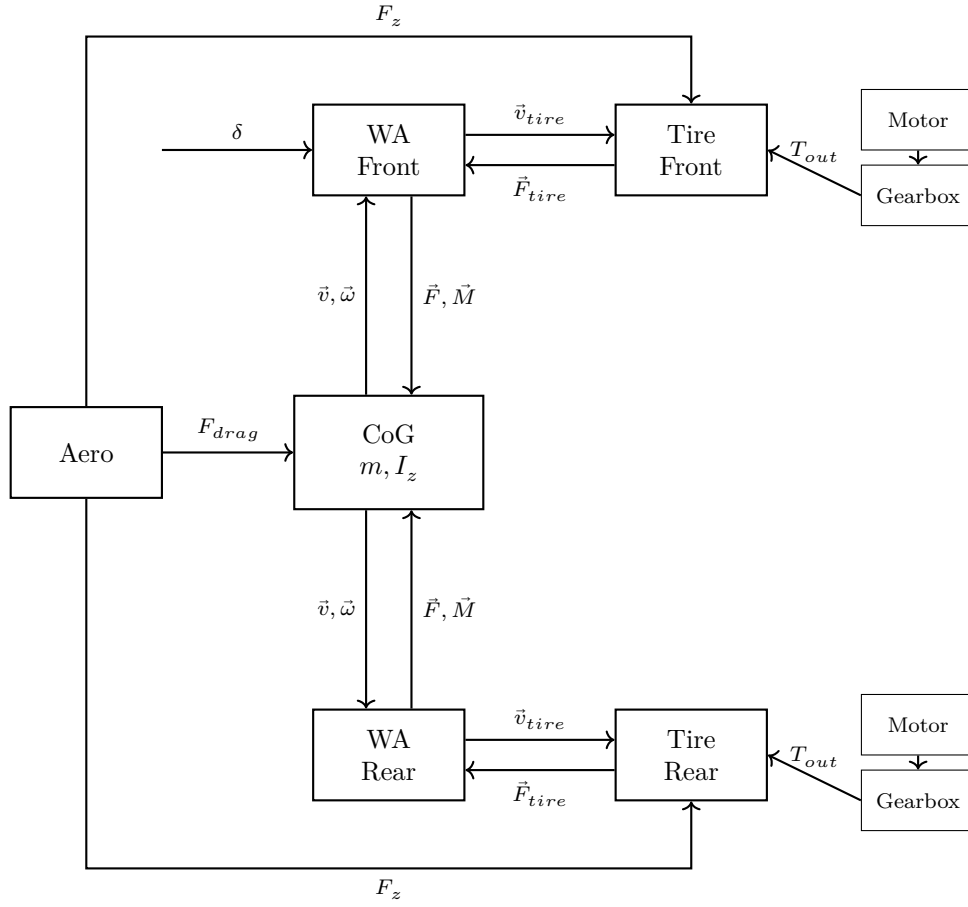


Figure 2.4: Single-track (bicycle) model computational graph. Velocity flows from CoG through wheel assemblies to tires, forces flow back. Motors provide torque through gearboxes, aerodynamics provides drag and normal forces.

2.11 Change of independent variable, time to path

When differential equations for vehicle movement are stated. For use in lap time simulation it is beneficial to describe the car states not in time but in distance traveled. This introduces connection with location on track and reduces number of state variables by one. This tranformation is described in[22].

$$\varepsilon = \psi - \theta(s) \quad (2.1)$$

$$S_f = \frac{dt}{ds} = \frac{1 - n C(s)}{v_x \cos \varepsilon - v_y \sin \varepsilon} \quad (2.2)$$

$$\frac{dn}{dt} = v_x \sin \varepsilon + v_y \cos \varepsilon \quad (2.3)$$

$$\frac{d}{ds} \begin{pmatrix} v_x \\ v_y \\ \psi \\ \omega \\ n \\ t \end{pmatrix} = S_f \begin{pmatrix} \dot{v}_x \\ \dot{v}_y \\ \dot{\psi} \\ \dot{\omega} \\ \dot{n} \\ 1 \end{pmatrix} \quad (2.4)$$

Chapter 3

The Optimal Control Problem

In previous chapter I have described how to build differential equations that govern movement of vehicle. Methods discussed in this section are doing essentially the same thing, solving controlled ODE along a time/path trajectory. But their approach and results are different.

3.1 Nonlinear programming

General NLP problems can be written as:

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & \mathbf{h}(\mathbf{x}) = 0 \\ & \mathbf{g}(\mathbf{x}) \leq 0 \end{aligned} \tag{3.1}$$

3.1.1 KKT conditions

alpha and omega of nonlinear programming, lord praise the KKT conditions.

$$\begin{aligned} \nabla f(\mathbf{x}) + \sum_{i=1}^m \lambda_i \nabla h_i(\mathbf{x}) + \sum_{i=1}^p \mu_i \nabla g_i(\mathbf{x}) &= 0 \\ \mathbf{h}(\mathbf{x}) &= \mathbf{0} \\ \mathbf{g}(\mathbf{x}) &\leq \mathbf{0} \end{aligned} \tag{3.2}$$

$$\begin{aligned} \mu_i &\geq 0, \quad i = 1, \dots, p \\ \mu_i g_i(\mathbf{x}) &= 0, \quad i = 1, \dots, p \end{aligned}$$

$$\begin{pmatrix} \nabla^2 \mathcal{L} & A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} -\Delta x \\ \lambda \end{pmatrix} = \begin{pmatrix} \nabla f(x) \\ c(x) \end{pmatrix} \tag{3.3}$$

3.2 Solving controlled ODE, initial value problems and boundary value problems

USE BOUNDARY AND INITIAL VALUE PROBLEM INSTEAD OF TIME MARCHING The most common method for solving controlled ODE is time

marching. Where the state computation is always for the states at time $t+1$, after solving at $t+1$, time is moved/marched forward and solution at $t+2$ is calculated and so on. This gives a solution to ODE. If the ODE can be controlled like the car, you also need a separate controller, which becomes part of the ODE. This approach gives a solution to the problem of how to control car, so that it completes a lap around a circuit. However, this solution is not optimal to time or any objective as the controls depend on the controller. Therefore, if the car parameters change and the controller doesn't, the resulting time can be exactly same, because controller cannot take advantage of lighter car. Sensitivity analysis and lap time can be obtained but it is skewed by the controller chosen. In Car maker the controller is driver and they have Driver adaptation. Using time marching is not best for sensitivity analysis and lap time simulation. Controller can be tuned after each change of model but, that is time intensive. Another approach to solving controlled ODE is using shooting methods. Now the algorithm doesn't compute the states at only next time point, but it computes all states at all time points at the same time. Using this method it is possible to get rid of the controller, and instead of control rule a feed forward control vector along the trajectory is computed. This chapter deals with transcribing the continuous problem into discrete steps

■ 3.2.1 Initial value problem

■ 3.2.2 Forward backward pass

DESCRIBE POPULAR LAPTIME SIMULATION METHOD THAT USES FORWARD AND BACKWARD PASS FOR A MODEL OF LOW COMPLEXITY

■ 3.3 Transcription methods

■ 3.3.1 RK methods

Most straightforward method to transcribe continuous optimal control problem into non linear program, is to use explicit Runge Kutta solving methods from initial value problem as equality constraints on the dynamics of vehicle. Using euler method, the constraints for the dynamics of vehicle:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + h \cdot f(\mathbf{x}_k, \mathbf{u}_k) \quad (3.4)$$

This works, but it may be the worst possible transcription method, due to high error and instability of euler method [?]. Problems of euler method encountered in time marching simulation also translate to nonlinear programming. Implicit RK methods can be used too, In Initial value problems they are used less often, as they require iterative solving. Optimal control problem is Boundary value problem and not initial value problem, therefore:

...for a boundary value problem ...any scheme becomes effectively implicit. Thus, the distinction between explicit and implicit initial value schemes becomes less important in the BVP context.

[4, p. 69]

For boundary value problem it makes no sense to use explicit Runge kutta methods, as they have inferior stability compared to implicit runge kutta methods.[23, p. 4]

For Boundary value problems commonly implicit runge kutta method used is from LobattoIIIA family, but any other method can be used. Book [6] uses only these.

3.3.2 Collocation methods

The concept of collocation is old and universal in numerical analysis ...For ordinary differential equations it consists in searching for a polynomial of degree s whose derivative coincides ("co-locates") at s given points with the vectorfield of the differential equation ...[2, p. 224]

Job of collocation method is to find a polynomial that approximates the solution of BVP along independent variable (most commonly time). Collocation methods are a subset of implicit runge kutta methods. The main distinction between runge kutta methods and collocation. Is that the result of collocation is always polynomial, that does not hold for general Runge Kutta method. Collocation methods are the core of this thesis, I have implemented Gauss radau, gauss, gauss lobatto. Collocation methods differ in points at which the derivative of polynomial coincides ("co-locates") with the vector field of differential equation. Commonly used methods in optimal control are radau, lobatto, legendre. They can be written in integral and differential form[11]. Collocation methods in this thesis are based on Framework described in[12]

Polynomial that approximates the solution of ODE is based on Legendre polynomials. Important note that in actual implementation this calculation of polynomial basis is impractical, therefore barycentric form from [5] has been used as implementation of these methods.

$$L_i(\tau) = \prod_{\substack{j=0 \\ j \neq i}}^N \frac{\tau - \tau_j}{\tau_i - \tau_j}, \quad i = 0, \dots, N \quad (3.5)$$

$$x_j^N(\tau) = \sum_{i=0}^N x_{ij} L_i(\tau) \quad (3.6)$$

the differentiation matrix is made also using barycentric form from [5, p.513] Many articles omit, that this barycentric form is crucial for fast code.

$$\dot{x}_j^N(\tau_k) = \sum_{i=0}^N x_{ij} \dot{L}_i(\tau_k) = \sum_{i=0}^N D_{ki} x_{ij}, \quad D_{ki} = \dot{L}_i(\tau_k) \quad (3.7)$$

then the output?? of differential equation can be aproximated using the differentiation matrix.

$$\dot{x}_j^N(\tau_i) = (D\mathbf{X})_{ij} \quad (3.8)$$

This is then used as constraints and voila nonlinear optimal control problem is born. 🤖

$$\begin{aligned} \min \quad & \Phi(\mathbf{X}_{N+1}) \\ \text{s.t.} \quad & D\mathbf{X} = F(\mathbf{X}_{\text{LG}}, \mathbf{U}_{\text{LG}}) \\ & \mathbf{X}_{N+1} = \mathbf{X}_0 + \mathbf{w}^T F(\mathbf{X}_{\text{LG}}, \mathbf{U}_{\text{LG}}) \\ & \mathbf{X}_0 = \mathbf{x}_0 \end{aligned} \quad (3.9)$$

■ 3.4 Mesh refinement

■ 3.4.1 collocation integral form

integral form is straight forward, same as other unge kutta Methods

■ 3.4.2 collocation differential form

differeanital form requires to create differentiation matrix of the aproximating polnomial,which contains the difference of the approximating polnomial.

■ 3.4.3 p adaptive methods

p adaptive methods work by increasing the order of polynomial, until the error is below user defined threshold. Figure ?? shows that error along track is not distributed equally. Error is localised around areas with large jump in controls. These areas need finer sampling. P methods cannot focus these specific points directly, they focus entire trajectory.

The errors are concentrated around the discontinuity of controls, where the behaviour of the vehicle is more complex/ requiring high order or finer approximation. Figure shows this??

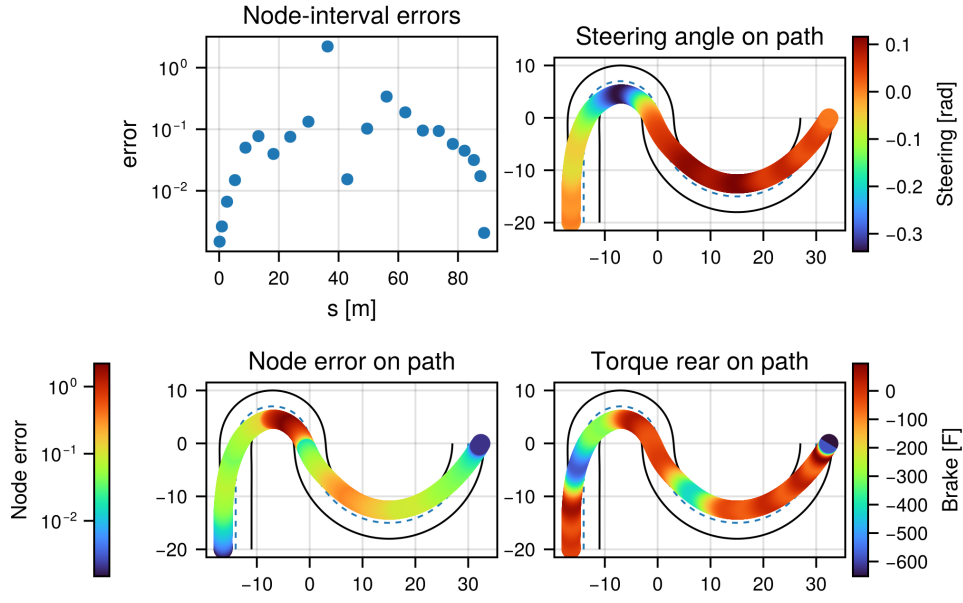


Figure 3.1: Node-interval ODE error for power-test case.

3.4.4 h adaptive methods

h methods are used when solution of optimal control problem is approximated by many intervals with low degree polynomial. These methods can decide to split segment into two smaller segments, they use different techniques to do this. Method described in [6] calculates error using DOPLNIT. Another method described in [16] uses techniques from essentially nonoscillatory (ENO) interpolation

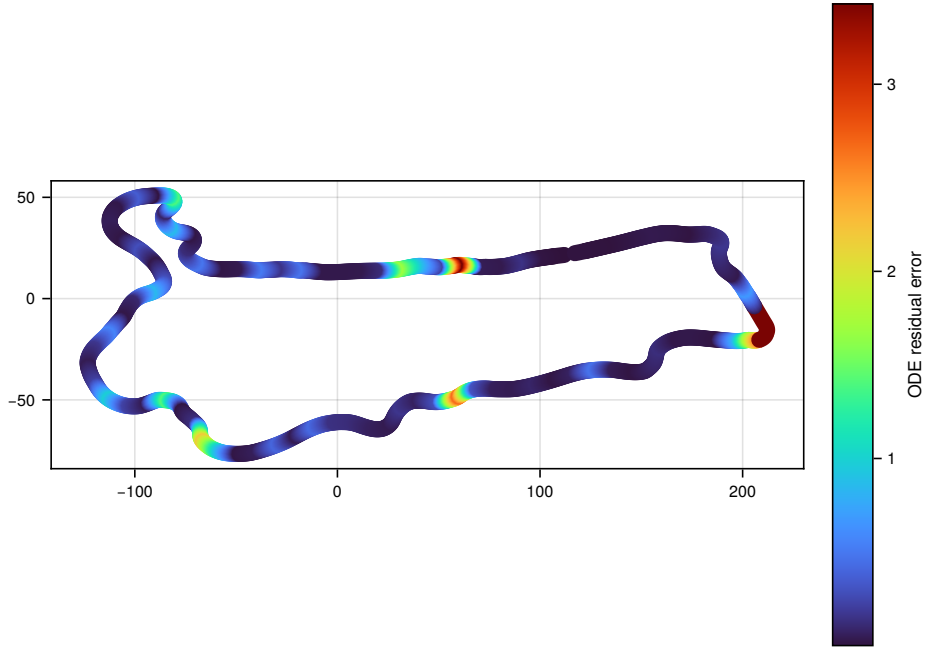


Figure 3.2: Node-interval ODE error for power-test case.

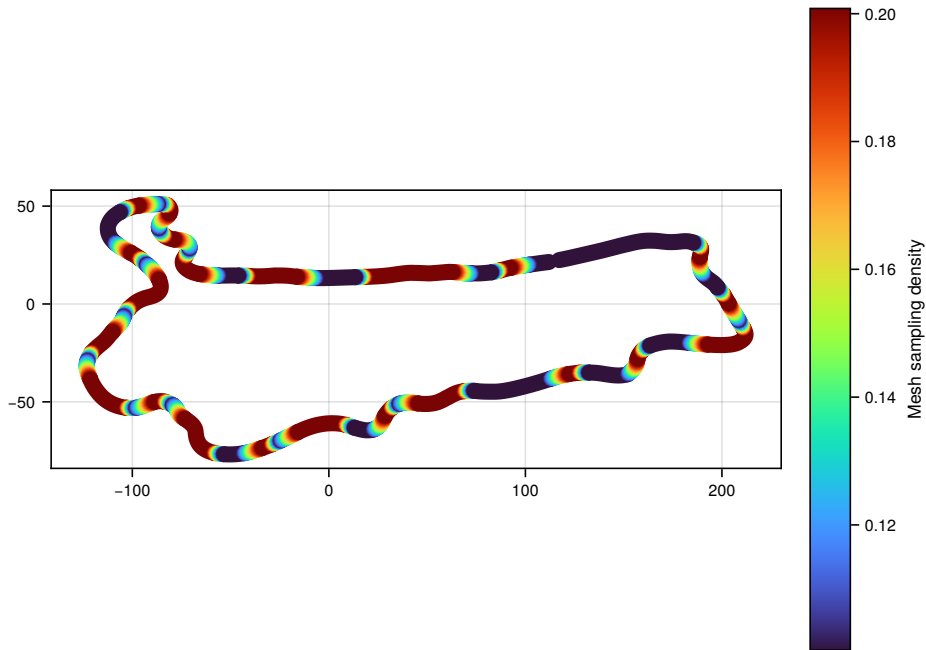


Figure 3.3: Node-interval ODE error for power-test case.

[14] to decide where to split and remove nodes.

■ 3.4.5 hp adaptive methods

hp adaptive methods are state of the art in mesh refinement. They are combination of increasing/decreasing mesh segments and increasing/decreasing order of approximating polynomial inside segments. One of the first hp-method and commonly cited is described in article [10] from University of Florida. They have produced numerous papers on hp- adaptive mesh refinement [3] [18]. Recent work on mesh refinement for functions with kinks [28].

■ 3.5 Adjoint methods

Adjoint methods allow for differentiating the solution of optimization problem. In this case it is function of time on states, controls and car parameters along the trajectory. This is very useful as it greatly simplifies sensitivity analysis. Traditional way of sensitivity analysis is to compute several runs with different vehicle parameters, but this allows to compute sensitivity of all parameters in one shot.



Part II

Implementation of Optimal Control problem

Chapter 4

Frameworks for Optimal Control Problem Formulation

Process of solving optimal control problem has 3 parts. Problem definition, solving of nonlinear program, optional mesh refinement

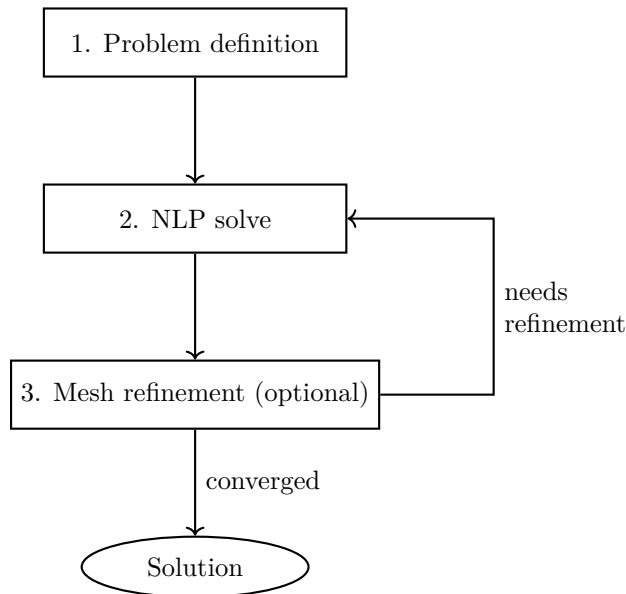


Figure 4.1: Optimal control problem solving pipeline.

The consensus in optimal control community about tools for problem formulation is that:

There is no shortage of optimal control software based on pseudospectral approaches. There are a number of other optimal control libraries that tackle similar kinds of problems, such as OTIS4, GPOPS-II, and CASADI.

[?][?] This chapter is about tool that define the OCP, and supply derivatives of the problem, they cannot solve the problem by themselves.

GPOPS-II [?] is perhaps the most advanced tool for optimal control, it was developed at University of Florida by Michael Patterson and Anil V. Rao. Other tools allow defining the problem, but GPOPS-II is a level higher. It also includes advanced mesh refinement strategies, multiple phase OCP and automatic scaling. It interfaces with MATLAB and C++, uses ADIGATOR [?] for automatic differentiation. This tool is commercial and closed source. Implementation in this thesis is heavily based on work of community from University of Florida.

CasADi is an open-source tool for nonlinear optimization and algorithmic differentiation.[?][?]. interfaces with C++,Python,MATLAB.

Python

[?]

ExaModels, JuMP

Chapter 5

NLP solvers, GPU exploitation???

Once the objective function and constraints are created, the problem is passed to NLP solver. Very popular methods are sequential quadratic solvers and interior point solvers.

5.1 Interior point methods: IPOPT, MadNLP

Interior point solvers or barrier solvers work by simplifying the NLP into a barrier problem and solving a sequence of barrier problems, with decreasing barrier parameter. Solver used in this thesis is IPOPT [29]. Article in [27] describes framework that unifies algorithmic components of different solvers, allowing for custom new solvers.

5.1.1 Barrier problem formulation

Article [29] Describes how to transform generic NLP into form 5.1 $h(\mathbf{x}) \geq 0$, interior point methods solve a barrier problem.

$$\begin{aligned} \min_{\mathbf{x} \in \mathbb{R}^n} \quad & \phi_\mu(\mathbf{x}) := f(\mathbf{x}) - \mu \sum_{i=1}^n \ln(x_i^{(i)}) \\ \text{s.t.} \quad & c(\mathbf{x}) = 0 \end{aligned} \tag{5.1}$$

5.2 Sequential quadratic programming

5.3 Hessian calculation through Algorithmic differentiation

Major complication of many NLP solvers is that they require Hessian of the objective function in order to compute the step. computation of it can be time intensive. Most universal method is finite difference, that means evaluating the function around current point and estimating the hessian. other method is using symbolic differentiation, often the full symbolic expression is too large to be evaluated effectively due to chain and product rule. Algorithmic

differentiation aims to solve problems of previous methods, by applying the chain rule to the elementary operations of a function. Comprehensive review of automatic differentiation is in article [9]

■ 5.4 Scaling and matrix conditioning

Common pitfall of nonlinear programming is poor scaling, this happens when the solver encounters variables, that are orders of magnitude apart, for example motor force in thousands of newton and heading angle in radians. This happens, because at the core of nonlinear programming solver is a linear solver (MUMPS, MA97, PARDISO,...), that has to do matrix factorization every iteration, to solve KKT linear system. If this linear system is ill conditioned due to improper scaling, the factorization may be imprecise and search direction corrupted.

Another thing which led to issues is constraints qualification. Linear independence constraint qualification requires gradients of the active constraints to be linearly independent, then the jacobian of the constraints is rank deficient and linear solve may fail

■ 5.5 sparsity

Sparsity is good, sparsity means matrix Hessians and Jacobians with many zeros also sparsity in constraints, that means not having super very long expression with many terms in constraint. I guess it is because derivative is difficult then and can lead solver in wrong direction or take small steps. Therefore using more decision variables to define same problem can actually lead to faster computation [17]

■ 5.6 Initialization

NLP solvers require initial guess for all decision variables, to start solving. For this problem initial guess is obtained driving the vehicle on given track. PD controller is used to keep the car on the centerline and P controller is used to keep velocity at 5m/s. In general, it is possible to initialize all variables at zero, but with large scale nonlinear programming this approach is not used. For this problem initialization with zeros leads to singularity in equation 2.2 and is not possible at all. must be initialized otherwise very bad?

■ 5.7 Global and local optimums

This optimization problem is generally non convex, therefore any local optimum does not guarantee global optimum. Commonly used NLP solvers, like IPOPT guarantee only local optimums and not global ones. Good initialization is basis for finding a local optimum close to the global optimum. Source?????

Chapter 6

My implementation, julia + JuMP

I have decided to use julia + JuMP, because I wanted to use free and open source software.

I have already done a lot in Python and julia seemed like new interesting thing to learn. Julia is supposed to be fast and high level language, bridging the gap between C++ and python. Julia packages offer a lot of ready to use mathematical tools developed by community.

JuMP [19] (Julia for Mathematical Programming) is a modelling language and collection of optimization packages in Julia. JuMP makes it possible to create decision variables and constraints symbolically and uses automatic differentiation, to supply first and second derivative to solvers. Solver used in this thesis is IPOPT [29] and MadNLP[26][25].

6.1 My optimization stack

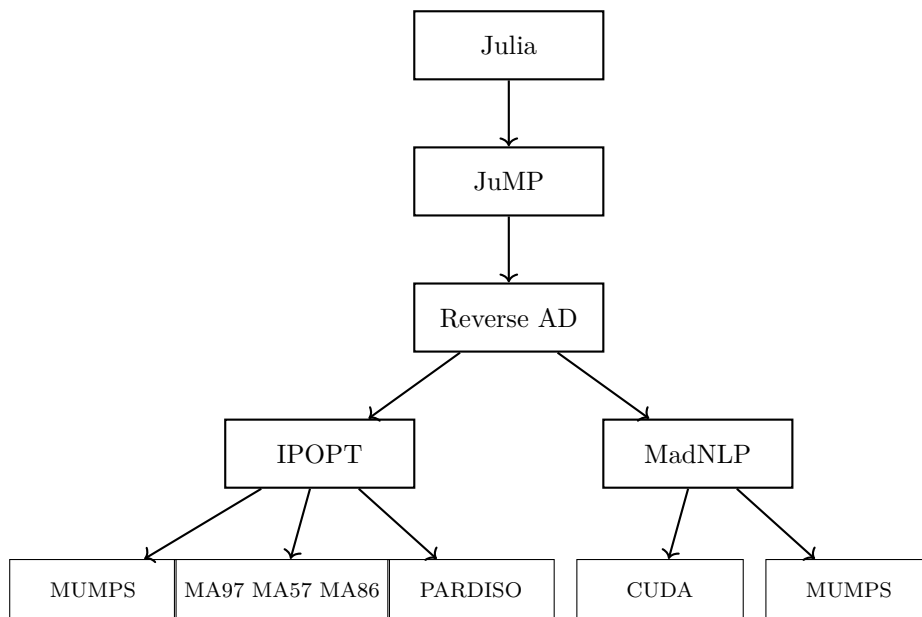


Figure 6.1: Optimization software stack

■ 6.2 Gauss methods as pseudospectral collocation

I have implemented Gauss-Legendre, Gauss-Lobatto, Gauss-Radau as global collocation methods, that is one polynomial approximating the entire trajectory. I have implemented them according to [11]

■ 6.3 Gauss methods as h-method

■ 6.4 Diving problem into many lower degree polynomials

■ 6.5 Mesh refinement methods



Part III

results

Chapter 7

comparison of methods

Performance of NLP solvers and frameworks is often tested on benchmark problems. like CUTEst [13] dataset?

Table 7.1: Benchmark summary statistics by transcription variant.

var <i>String</i>	runs <i>Int64</i>	ok <i>Int64</i>	fail <i>Int64</i>	rate <i>Float64</i>	mean_s <i>Float64</i>	med_s <i>Float64</i>	min_s <i>Float64</i>	max_s <i>Float64</i>
Legendre	15	10	5	66.6667	96.9307	91.0846	10.2008	245.1052
Lobatto IIIA	15	10	5	66.6667	115.1588	108.4854	5.4434	298.3367
Radau	15	11	4	73.3333	84.0252	55.7858	5.4386	213.1315

Table 7.2: Benchmark results by track, vehicle model, and transcription variant.

track <i>String</i>	car <i>String</i>	var <i>String</i>	ok <i>String</i>	status <i>String</i>	time_s <i>Float64</i>	obj <i>Float64</i>
FSCZ	singletrack	Radau	yes	solved	34.9547	50.2106
FSCZ	singletrack	Legendre	yes	solved	120.2695	50.5890
FSCZ	singletrack	Lobatto IIIA	yes	solved	105.9758	51.4710
FSCZ	twintrack	Radau	yes	solved	213.1315	57.3381
FSCZ	twintrack	Legendre	no	locally infeasible	492.7919	NaN
FSCZ	twintrack	Lobatto IIIA	no	locally infeasible	404.2764	NaN
FSCZ	bus	Radau	no	error	NaN	NaN
FSCZ	bus	Legendre	no	error	NaN	NaN
FSCZ	bus	Lobatto IIIA	no	error	NaN	NaN
Berlin	singletrack	Radau	yes	solved	83.3507	79.4234
Berlin	singletrack	Legendre	yes	solved	103.5907	79.4033
Berlin	singletrack	Lobatto IIIA	yes	solved	159.4399	79.1299
Berlin	twintrack	Radau	yes	solved	150.8455	77.7439
Berlin	twintrack	Legendre	yes	solved	245.1052	77.7883
Berlin	twintrack	Lobatto IIIA	yes	solved	198.3435	77.8983
Berlin	formula	Radau	yes	solved	112.1158	87.9477
Berlin	formula	Legendre	yes	solved	126.9144	88.3235
Berlin	formula	Lobatto IIIA	yes	solved	173.1226	87.9482
FSG	singletrack	Radau	yes	solved	45.0607	58.2463
FSG	singletrack	Legendre	yes	solved	78.5785	58.1609
FSG	singletrack	Lobatto IIIA	yes	solved	110.9951	56.8773
FSG	twintrack	Radau	yes	solved	187.9523	69.3149
FSG	twintrack	Legendre	yes	solved	197.7527	70.3348
FSG	twintrack	Lobatto IIIA	yes	solved	298.3367	69.8878
FSG	bus	Radau	no	error	NaN	NaN
FSG	bus	Legendre	no	error	NaN	NaN
FSG	bus	Lobatto IIIA	no	error	NaN	NaN
DoubleTurn	singletrack	Radau	yes	solved	5.4386	5.7087
DoubleTurn	singletrack	Legendre	yes	solved	10.2008	5.6794
DoubleTurn	singletrack	Lobatto IIIA	yes	solved	5.4434	6.0247
DoubleTurn	twintrack	Radau	yes	solved	26.5157	6.5283
DoubleTurn	twintrack	Legendre	yes	solved	11.4406	6.5835
DoubleTurn	twintrack	Lobatto IIIA	yes	solved	21.6274	7.0205
DoubleTurn	bus	Radau	no	error	NaN	NaN
DoubleTurn	bus	Legendre	no	error	NaN	NaN
DoubleTurn	bus	Lobatto IIIA	no	error	NaN	NaN
FigureEight	singletrack	Radau	yes	solved	9.1259	14.9127
FigureEight	singletrack	Legendre	yes	solved	16.2593	14.9162
FigureEight	singletrack	Lobatto IIIA	yes	solved	15.4936	14.8296
FigureEight	twintrack	Radau	yes	solved	55.7858	16.5690
FigureEight	twintrack	Legendre	yes	solved	59.1957	16.5736
FigureEight	twintrack	Lobatto IIIA	yes	solved	62.8097	16.5319
FigureEight	bus	Radau	no	error	NaN	NaN
FigureEight	bus	Legendre	no	error	NaN	NaN
FigureEight	bus	Lobatto IIIA	no	error	NaN	NaN



Chapter 8

Conclusion



Appendices



Appendix A

Bibliography

- [1] A Computationally Efficient Framework for Free-trajectory Minimum-lap-time Optimization of Racing Cars. <https://arxiv.org/html/2511.13522v1>.
- [2] Solving ordinary differential equations I. nonstiff problems. *Mathematics and Computers in Simulation*, 29(5):447, October 1987.
- [3] Yunus Agamawi and Anil Rao. CGPOPS: A C++ Software for Solving Multiple-Phase Optimal Control Problems Using Adaptive Gaussian Quadrature Collocation and Sparse Nonlinear Programming. *ACM Transactions on Mathematical Software*, 46, May 2020.
- [4] Uri M. Ascher, Robert M. M. Mattheij, and Robert D. Russell. *Numerical Solution of Boundary Value Problems for Ordinary Differential Equations*. Classics in Applied Mathematics. SIAM, 1995.
- [5] Jean-Paul Berrut and Lloyd N. Trefethen. Barycentric Lagrange Interpolation. *SIAM Review*, 46(3):501–517, January 2004.
- [6] John T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. Society for Industrial and Applied Mathematics, second edition, January 2010.
- [7] Forbes T. Brown. *Engineering System Dynamics: A Unified Graph-Centered Approach, Second Edition*. CRC Press, Boca Raton, 2 edition, August 2006.
- [8] Richard Ciglanský. Analysis of vehicle components and their impact on lap-time.
- [9] Daniel Cortild, J Haastert, A Sanabria, and F Voronine. *A Brief Review of Automatic Differentiation*. March 2023.
- [10] Christopher L. Darby, William W. Hager, and Anil V. Rao. An hp -adaptive pseudospectral method for solving optimal control problems. *Optimal Control Applications and Methods*, 32(4):476–502, July 2011.
- [11] Divya Garg, Michael Patterson, William Hager, Anil Rao, David R Benson, and Geoffrey T Huntington. An overview of three pseudospectral methods for the numerical solution of optimal control problems.

- [12] Divya Garg, Michael Patterson, William W. Hager, Anil V. Rao, David A. Benson, and Geoffrey T. Huntington. A unified framework for the numerical solution of optimal control problems using pseudospectral methods. *Automatica*, 46(11):1843–1851, November 2010.
- [13] Nick Gould, Dominique Orban, and Philippe Toint. CUTEst: A Constrained and Unconstrained Testing Environment with safe threads for Mathematical Optimization. *Computational Optimization and Applications*, 60, July 2014.
- [14] A. Harten, B. Engquist, S. Osher, and S. R. Chakravarthy. Uniformly high order accurate essentially non-oscillatory schemes, III. *Journal of Computational Physics*, 131(1):3–47, 1997.
- [15] Himanshu Hatwal and E. C. Mikulcik. Some inverse solutions to an automobile path-tracking problem with input control of steering and brakes. *Vehicle System Dynamics*, 15:61–71, 1986.
- [16] Shuang Li* Jisong Zhao. Adaptive mesh refinement method for solving optimal control problems using interpolation error analysis and improved data compression.
- [17] Julia Discourse. 90% of IPOPT run time is spent in Hessian-constraints evaluation, very little time in linear solver — is it normal? <https://discourse.julialang.org/t/90-of-ipopt-run-time-is-spent-in-hessian-constraints-evaluation-very-little-time/136708>, 2025. Accessed: 2026-04-19.
- [18] Fengjin Liu, William W. Hager, and Anil V. Rao. Adaptive Mesh Refinement Method for Optimal Control Using Decay Rates of Legendre Polynomial Coefficients. *IEEE Transactions on Control Systems Technology*, 26(4):1475–1483, July 2018.
- [19] Miles Lubin, Oscar Dowson, Joaquim Dias Garcia, Joey Huchette, Benoît Legat, and Juan Pablo Vielma. JuMP 1.0: Recent improvements to a modeling language for mathematical optimization. *Mathematical Programming Computation*, 2023.
- [20] Matteo Massaro and David Limebeer. Minimum-lap-time optimisation and simulation. *Vehicle System Dynamics*, 59:1–45, April 2021.
- [21] Yukio Nakajima. Advanced tire mechanics. 2019.
- [22] Giacomo Perantoni and David J.N. Limebeer. Optimal control for a Formula One car with variable parameters. *Vehicle System Dynamics*, 52(5):653–678, May 2014.
- [23] Anil Rao. A Survey of Numerical Methods for Optimal Control. *Advances in the Astronautical Sciences*, 135, January 2010.

- [24] R. D. Roland and C. F. Thelin. Computer simulation of Watkins Glen grand prix circuit performance. Technical Report ZL-5002-K-1, Cornell Aeronautical Laboratory, 1971.
- [25] Sungho Shin, Mihai Anitescu, and François Pacaud. Accelerating optimal power flow with GPUs: SIMD abstraction of nonlinear programs and condensed-space interior-point methods. *Electric Power Systems Research*, 236:110651, 2024.
- [26] Sungho Shin, Carleton Coffrin, Kaarthik Sundar, and Victor M Zavala. Graph-based modeling and decomposition of energy infrastructures. *IFAC-PapersOnLine*, 54(3):693–698, 2021.
- [27] Charlie Vanaret and Sven Leyffer. *Implementing a unified solver for nonlinearly constrained optimization*. November 2025.
- [28] Hendrik Wilka and Jens Lang. Adaptive hp-Polynomial Based Sparse Grid Collocation Algorithms for Piecewise Smooth Functions with Kinks, May 2025.
- [29] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, March 2006.